

Programmeringsparadigmer 2025

Kursusgang 2: Første skridt

Hans Hüttel

16. september 2025

Vores plan for i dag

1. Hvordan vi udnytter vores tid her.
2. Læringsmålene
3. Forberedelsesopgaverne
4. Noget om biblioteker
5. En diskussion
6. Opgave nr. 1
7. Pause
8. Opgave nr. 2
9. Endnu en diskussion
10. Opgave nr. 3
11. Hvis tiden tillader det: Flere opgaver i dit eget tempo.
12. Vi evaluerer dagens kursusgang – **bliv venligst til slut!**

En advarsel

Denne kursusgang er ikke særlig ambitiøs. Det er helt bevidst – vi skal sikre os, at opsætningen virker, så de næste kursusgange kan udnytte den på en god måde.

De følgende kursusgange vil være mere krævende.

Læringskurven for dette kursus

Første halvdel af kurset

Det grundlæggende:

Typer, lister,
abstraction,
listeabstraktion,
højereordens-
funktioner...

Anden halvdel af kurset

*Hvordan man
strukturerer software:*
I/O, funktorer,
monader,
continuations, lazy
evaluering

Hvordan vi udnytter vores tid her

Vi har fire hovedaktiviteter:

- ▶ At tale om forberedelsesopgaverne
- ▶ Diskussioner om typiske fejl og misforståelser: Her er et stykke ”interessant kode”. Hvad er der galt med den? **Jeg vil bruge tavlen til dette.**
- ▶ At skrive ny kode: Problemløsning i par.
- ▶ En diskussion om de udfordringer, vi støder på i dag.

Vær rar at tage noter undervejs. I er deltagere, ikke tilskuere. Hjælpelærerne og jeg er altid til rådighed.



Deling af kode fra løsninger

Der er en fælles mappe til denne kursusgang, hvor I må uploade løsninger, så vi kan præsentere dem.

Der vil være en mappe af denne slags til hver af de følgende kursusgange.

Husk venligst at skrive jeres navn(e) i filen.

Læringsmål

Læringsmålene er:

- ▶ At kunne forklare begrebet funktion i det funktionelle programmeringsparadigme på en præcis måde
- ▶ At forstå de centrale punkter i den funktionelle programmerings historie
- ▶ At kunne installere Glasgow Haskell Compiler og bruge den med et simpelt programmeringsmiljø
- ▶ At kunne skrive simple definitioner i Haskell
- ▶ At kunne redigere, indlæse og bruge Haskell-programmer
- ▶ At forstå og kunne anvende simple aspekter af Haskell-syntaks: Definitioner, kommentarer, layout-reglen og **where**-deklarationer i definitioner.

Forberedelsesopgave – Afprøvning af Haskell (5 minutter)

Indlæs programmet [simple.hs](#), som er tilgængeligt fra Moodle-sektionen om denne kursusgang.

- ▶ Prøv at evaluere `laengde myList`.
- ▶ Hvad tror I resultatet af `sumtree myBigOak` bliver? Prøv at forklare hvorfor.
- ▶ Tjek derefter jeres svar ved at spørge Haskell.

Forberedelsesopgave – Det andet element (5 minutter)

Brug funktionerne i afsnit 2.4 til at definere en funktion `second`, som, når den får en liste `xs`, returnerer det andet element i `xs`, hvis det findes. Som eksempler på, hvad denne funktion skal gøre, forventer vi, at

```
second [1,4,5,6]
```

giver os `4`, og at

```
second ["some", "bizarre", "mango"]
```

giver os `"bizarre"`. Vis, at jeres definition af `second` virker for disse eksempler på argumenter. Find derefter to flere eksempler på argumenter og se, hvad der sker. Er jeres funktion en total funktion?

Næste uge

Der er også forberedelsesopgaver næste gang.

Hvem vil være med til at fortælle, hvad de har lavet?

Noget om biblioteker

Ethvert problem, I bliver bedt om at løse i dette kursus, kan løses ved **kun at bruge de funktioner, der er defineret i Haskell-præludiet**. Spild ikke jeres tid på at lede efter funktioner i obskure Haskell-biblioteker, der kan løse problemet.

Husk, hvad den tid, vi bruger her, er beregnet til: Den er der for at hjælpe jer med at få erfaring, så I kan nå læringsmålene.

Opgaverne er en ramme, hvor I kan få erfaring – de er ikke problemer, der opstår af et behov, så ”med alle midler” gælder ikke her.

Jeres spørgsmål

Mens I forberedte jer til denne kursusgang, har I kunnet overveje hvilke spørgsmål, det var vigtigst for jer at få svar på.

Hvad er de vigtigste spørgsmål, I har netop nu?

En diskussion (5 minutter)

En berømt influencer postede et kodeudsnit i Haskell på Instagram. Her er det:

```
x = 4
```

```
x = x + 4
```

Influenceren havde meget erfaring med C og kunne ikke forstå, hvorfor et så simpelt stykke kode ikke ville virke her. ”Det virker fint i C – dette giver ingen mening!” skrev influenceren.

Kan I hjælpe influenceren **uden at spørge Haskell**?

Opgave 1 (10 minutter)

Definer en funktion `allbutsecond`, som, når den får en liste `list`, returnerer listen bestående af alle elementer undtagen det andet element i listen. Som eksempler på, hvad denne funktion skal gøre, forventer vi, at

```
allbutsecond [1,4,5,6]
```

skal give os `[1,5,6]`, og at

```
allbutsecond ["some", "bizarre", "mango"]
```

vil give os `["some", "mango"]`.

Hvordan kan I blive mere sikre på, at jeres løsning er korrekt?

En pause

Opgave 2 (20 minutter)

Definer en funktion `midtover`, som, når den får en liste `xs` af længde n , returnerer et par af to lister `(xs1,xs2)`, så `xs1` består af de første $\lfloor \frac{n}{2} \rfloor$ elementer fra `xs`, og så `xs2` består af de resterende elementer.

```
midtover [1,4,5,6]
```

skal give os `([1,4],[5,6])` , og at

```
midtover ["this", "is", "actually", "a", "fairly", "long", "list"]
```

vil give os

```
(["this", "is", "actually"], ["a", "fairly", "long", "list"])
```

Hvordan kan I blive mere sikre på, at jeres løsning er korrekt?

Endnu en diskussion (10 minutter)

I dagens tekst ser vi, at der ikke gøres noget for at tjekke, om argumentet til en funktion på heltal faktisk er et heltal.

Er dette noget, vi skal bekymre os om?

Opgave 3 (15 minutter)

Der er noget galt i følgende lille stykke kode. Men hvad er galt?
Forklar. Ret derefter koden, så den virker.

```
N = a 'div' length xs
  where
    a = 10
    xs = [1,2,3,4,5]
```

Endnu en diskussion

Forestil jer, I bliver bedt om at definere funktioner `iseven` og `isodd` med typerne

```
iseven :: Integral a => a -> Bool
```

```
isodd  :: Integral a => a -> Bool
```

så `iseven` tager et heltal og fortæller os, om det er et lige tal, og så `isodd` tager et heltal og fortæller os, om det er et ulige tal. Nogen kom med følgende løsning:

```
iseven n = if n `div` 2 == 0 then True  else  
            False
```

```
isodd n = if iseven n then False else True
```

Virker disse funktioner som tiltænkt? Er dette et eksempel på god kodningsstil i Haskell?

Arbejde i dit eget tempo (parprogrammering)

Der er flere opgaver i opgavesættet, som I kan arbejde med i resten af kursusgangen. Prøv at løse en eller flere af dem.

Evaluering

- ▶ Hvad fandt I svært?
- ▶ Hvad overraskede jer?
- ▶ Hvad gik godt?
- ▶ Hvad kunne vi forbedre? (F.eks. hvordan vi kan organisere aktiviteter i lokalet.) Vær opmærksom på, hvad der er realistisk for os alle.